CS 5010 Programming Design Paradigm
Fall 2023

Class Details

Class Hours:	CS 5010 Lecture 1:00-4:20 pm Tuesdays RXI 138
	CS 5011 Recitation for 5010 1:00-2:30 pm Wednesdays in RXI 132
Prerequisites:	N/A
Modality:	Traditional Lecture

Instructor Information

Name:	Dr. Brianna Dym
Email:	b.dym@northeastern.edu
Office:	Virtual
Office Hours:	By Appointment

Course Description

CS 5010 provides an intensive tour of programming and design. The main objective of the course is expose students to different ways of thinking of a problem and designing robust solutions using a mix of program design paradigms. We will discuss and compare object-oriented and functional design paradigms. We will concentrate on object-oriented program design and the design of abstractions that support the design of reusable software and libraries.

The course reviews typical object-oriented concepts such as information hiding, encapsulation and various forms of polymorphism. It contrasts the use of inheritance and composition as dual techniques for software reuse. It provides a deeper understanding of object-oriented design using the use of graphical design notations such as UML and object-oriented design patterns. Finally it emphasizes on testing, specifically unit testing of components.

An important aspect of this course is to write clean, well-structured, well-documented code. Students will be expected to design and program to facilitate reuse, minimize code duplication and make it "future-proof." Students will also be expected to explain and defend their design and programming decisions to code panels. In this sense this course trains students to see programming as a "people's discipline."

The course is suitable for all graduate students in the MSCS program. It assumes that students have prior experience in programming and design.

"Think first, experiment later."

The curiosity to ask questions, try out new things and the motivation to go above and beyond is essential to extract maximum benefit from this course. This is a programming-intensive course, but also emphasizes careful design and thorough testing.

Learning Goals

Upon completion of this course, a student will be able to:

- Design object-oriented solutions to small and moderately-sized problems
- Describe and implement design patterns appropriately
- Design and implement a clean, understandable and well documented solution in the Java programming language
- Defend choices made in design and implementation to a group of peers
- Generate appropriate documentation for developed solutions
- Design unit tests for a given component and implement them using the JUnit testing framework for Java programs
- Create, refine and express a design in graphical notation such as UML diagrams

- Explore existing documentation to describe and use existing libraries and frameworks

In order for this course to be a meaningful learning experience, you need to come to each class with all assigned readings and videos complete, as well as any due assignments finished to the best of your ability. If you do not complete the pre-class work, you will have a hard time following the in-class activities.

Course Assessment

Final grades will reflect students' effort and performance. The course grade will be based on the following categories:

Quizzes - 15%
Lab Assignments - 40%
Participation (measured by completing in-class activities) - 15%
Mid-Semester Project - 10%
Final Project - 20%

If class does not happen one week (e.g. because of a federal holiday), there will be alternative turn-in times for assignments.

At two points during the semester, you will build a project that demonstrates your understanding of overall program design. Assignments will be graded in terms of whether or not they meet the specified expectations. Each assignment will have an accompanying description of expectations. If your assignment does not meet expectations by the time it is due, you will need to continue working on it until it does meet expectations.

Late/Makeup Policy

All assignments have a specific due date and time.

- In-class exercises cannot be made up.
- If you are not going to submit something on time, or if you miss a deadline, you must talk to me about what you missed and what plan we can come up with to ensure you turn your work in soon and stay on track in class. Those are my requirements for making up absent or late work, and they are designed to mirror processes in professional environments.

This class is designed to have enough flexibility in its scoring that you can miss some class or do poorly on a few assignments and still succeed. However, if a major life event is preventing you from consistently turning in work on time, please reach out to your instructor to discuss alternative work methods, due dates, and potential solutions to navigating those challenges.

Grade Assessment

Your grade is reflected in Canvas, though if you have questions please email the instructor, not your TAs. I typically do not round up or offer extra credit. Our grading scheme will be:

A (93%)
A- (90%)
B+ (87%)
B (83%)
B- (80%)
C+ (77%)
C (83%)
C- (70%)
D (<70%)
F (<60%)

Course Materials

The required materials for this course are provided through Canvas. Students are expected to read each module's materials as well as view any of the supplemental videos before attempting that module's assignments.

Additional reading may be recommended utilizing Northeastern's subscription to the O'Reilly Learning Platform. You will be able to login using your "northeastern.edu" credentials.

https://learning.oreilly.com

A schedule of topics is available on Canvas.

Computing Environment

You will want to download and install an integrated development environment (IDE) to write your programs this semester. The instructor uses IntelliJ IDEA. The first lab assignment for the week will involve setting up a developer environment.

Handing in Assignments

Assignments will be handed in by uploading files to Canvas. There will be a named assignment submission area with instructions and assignment requirements. Review each assignment sheet for requirements.

Respect for Diversity

You belong in this class. Yes, you! If you ever feel as though something is beyond your understanding or you have no idea what is being discussed, please ask questions. Programming and computer science in general is a discipline that has been messily assembled piece by piece rather hastily in recent decades. Untangling and making sense of best practices, code requirements, and methods of design is an exercise in patience.

Academic Accommodations

If you have a documented need for academic accommodation, please contact the instructor within the first two weeks with documentation. Reach out to the Disability Resource Center to arrange for appropriate accommodations if you have not.

Mental health issues are real and can prevent you from doing your best work. Your Khoury advisor is your primary contact for accessing University resources. Do not hesitate to make use of them as needed. Please do not wait until it has seriously impacted your work.

Academic Integrity

Computer science, both academically and professionally, is a collaborative discipline. In any collaboration, however, all parties are expected to make their own contributions and to generously credit the contributions of others. In our class, therefore, collaboration on homework and programming assignments is encouraged, but you as an individual are responsible for understanding all the material in the assignment and doing your own work. Always strive to do your best, give generous credit to others, start early, and seek help early from both your professors and classmates.

The following rules are intended to help you get the most out of your education and to clarify the line between honest and dishonest work. The professor reserves the right to ask you to verbally explain the reasoning behind any answer or code that you turn in and to modify your project grade based on your answers.

If you have had a substantive discussion of any homework or programming solution with a classmate, then be sure to cite them in your explanation of the code. You will not be penalized for working together.

The following rules apply to anything you hand in for a grade:

- Do not copy substantive code directly from online sources. Looking up how basic syntax is formatted is fine, but if you start copying more complex, multi-line solutions it can lead to confusion and break parts of your program. This is not allowed.
- You may view each other's code and explain how you worked through a solution, but please provide a comment in the code where you were working together with someone.

- Do not rely on online discussion spaces like StackOverflow or ChatGPT. You will receive partial and conflicting solutions that are not situated in the understanding that we are building in this class. If you end up taking inspiration from one of these sources, you need to include comments in your code that explain what you implemented from ChatGPT or from someone's online discussion. Include a link to where you found the reference for your code.

The university's academic integrity policy discusses actions regarded as violations and consequences for students:

Office of Student Conduct and Conflict Resolution - Academic Integrity Policy

If I find that a significant part of your assignment is copied or downloaded from a source without prior approval from me, then your assignment will be submitted to the Academic Integrity Committee.

The Final Say on Academic Integrity: Should I discover that you have submitted any content that you did not work on *as your own work*, then it will receive a 0. This includes AI generated content and content that you paid for.

Anti-Discrimination and Harassment Policy

Title IX of the Education Amendments of 1972 protects individuals from sex or gender-based discrimination, including discrimination based on gender-identity, in educational programs and activities that receive federal financial assistance.

Northeastern's Title IX Policy prohibits Prohibited Offenses, which are defined as sexual harassment, sexual assault, relationship or domestic violence, and stalking. The Title IX Policy applies to the entire community, including male, female, transgender students, faculty and staff.

If you or someone you know has been a survivor of a Prohibited Offense, confidential support and guidance can be found through University Health and Counseling Services staff and the Center for Spirituality, Dialogue, and Service clergy members. By law, those employees are not required to report allegations of sex or gender-based discrimination to the University.

Alleged violations can be reported non-confidentially to the Title IX Coordinator within The Office for University Equity and Compliance at: titleix@northeastern.edu
and/or through NUPD (Emergency 617.373.3333; Non-Emergency 617.373.2121). Reporting Prohibited Offenses to NUPD does NOT commit the victim/affected party to future legal action.

Faculty members are considered "responsible employees" at Northeastern University, meaning they are required to report all allegations of sex or gender-based discrimination to the Title IX Coordinator.

Please visit the Office for University Equity and Compliance for a complete list of reporting options and resources both on- and off-campus.

Recording Policy

This course, or parts of this course, may be recorded for educational purposes. These recordings will be made available only to students enrolled in the course, the instructor of record, and any teaching assistants assigned to the course. If you have some objection to this, please see me ASAP.

Only students who have arranged an accommodation with the Disability Resource Center may use mechanical or electronic transcribing, recording, or communication devices in the classroom. Students with disabilities who believe they may need such an accommodation may contact the Disabilities Resource Center

Schedule of Topics

Week 1: Introduction to Design

September 12 - Lecture Topic: Java Review, Intro to Design Paradigms
September 13 - Lab Topic: IDE setup, verification, & Assignment 1 Introduction
Design patterns discussed: MVC

Week 2: Designing for Modularity

September 19 - Lecture Topic: Program Planning and Objects
September 20 - Lab Topic: Revising Assignment 1 progress, planning future iterations
Design patterns discussed: MVC, lookup techniques

Week 3: Responsibility & Data Driven Design

September 26 - Lecture Topic: Applying Design Strategies, Interface and Class Design
September 27 - Lab Topic: Introducing The Brewery Problem

Week 4: Abstraction

October 3 - Lecture Topic: Designing for Reuse
October 4 - Lab Topic: The Brewery Problem Continued

Week 5: Categorical Data Types

October 10 - Lecture Topic: Leveraging Complex Data Types in Design
October 11 - Lab Topic: Implementing JUnit Tests

Week 6: Data Structures

October 17 - Lecture Topic: Arrays, Maps, and More
October 18 - Lab Topic: Mid-Semester Project Work Time

Week 7: Behavioral Design Patterns

October 24 - Lecture Topic: How a Cockpit Remembers its Speed & Constructing Behaviors
October 25 - Lab Topic: Mid-Semester Project Work Time

Week 8: Tree Structures

October 31 - Lecture Topic: Binary Search Trees & Tree Data Structures
November 1 - Lab Topic: Escape the Maze

Week 9: Creational Design Patterns

October 7 - Lecture Topic: Factories & Builders
November 8 - Lab Topic: Generating the Maze

Week 10: Structural Design Patterns

November 14 - Lecture Topic: Adapters and Facades as Translators

November 15 - Lab Topic: Coding Telephone

Week 11: Event-Driven Design

November 21 - Lecture Topic: Event-Driven Programming and Design

November 22 - NO LAB. Fall Break

Week 12: GUIs

November 28 - Lecture Topic: Embodied Interaction

November 29 - Lab Topic: Fitts' Law

Week 13: Architectural Design Patterns

December 5 - Lecture Topic: MVC Revisited & MVVM

December 6 - Lab Topic: [new]

Week 14: Concurrency Patterns

December 12 - Lecture Topic: Threading, Monitoring, Reacting

December 13 - Individual Project Work Time